COMPANY CONFIDENTIAL

Machine Organization
Technical Memo #24

YORKTOWN
Research

April 11, 1961

Note on Non-Numerical Data Processing

by

H. Fleisher

Data processing by machine has been extended into an area commonly referred to as "non-numerical." What is meant by this phrase, evidently, is that the end result is not necessarily a number or group of numbers, but rather, it is a statement, or set of statements, or a document, or a name of a document; that is, an indication of a result obtained by primary use of logical operations on data rather than arithmetical operations. The basic logical operation is that of deductive inference, and the logical operators used in the framework of deductive inference depend on the data being processed. They must have a common property, namely, that of transitivity. * For example, the relations involved in determining the structure of a long chain molecule are those of class inclusion and identification of the lefthand member of a fragment of the molecule. (The latter defines a class). It can be seen that both of these relations are transitive.

The relations involved in recognizing characters of an alphanumeric system or in identification of members of a discrete set of objects are also transitive. They comprise class inclusion, plus relations that define classes.

There are many other examples of this type of data processing: it suffices that each can be generalized as described above.

Although this type of generalization involves discreteness among objects, as well as among properties, a real problem may originally have been stated such that properties have a continuous range of values, and different objects may have considerable overlap throughout these ranges of values of properties. This may be posed as a problem of analysis - similar to replacing differentials by differences in problems involving differential equations.

However, this analysis immediately brings in the crux of these problems: namely, their combinatorial nature. In fact, some of these problems become factorial in nature, with n! being of the order of 64! .

*If we let a, b, c be elements of the data set, and R a relation, and if we have (aRb) (bRc), such that the deductive conclusion is aRc, then R is transitive.

A simple example will illustrate this point. Consider character recognition of a single font. Assume that we have 26 alphabet characters, 10 numerical characters, and 10 special characters. This totals 46 characters to be identified. The extremes of the range of properties needed to identify individual characters can be specified in a simple way: (a) if each character had a unique property, we would have 46 unique properties to work with; (b) if each character had a unique set of properties, then the minimal number, m, of properties to be used to obtain 46 disjoint sets is $2^m \approx 46$, or $m \approx \log_2 46$. To simplify this discussion, we assume that we are dealing with distinct properties, so m is the nearest integer $> \log_2 46$.

To generalize, let P = number of properties, C = number of characters, $m \geq \log_2 C$, then $\log_2 C \leq P \leq C$. This range on P clearly permits a tremendously large variation of design parameters in a hardware situation, or of solution algorithms in a problem solving situation.

We must consider the lower limit of the range, $\log_2 C$, to be an ideal, not to be reached in practice, since it would minimize redundancy, and enhance the effects of errors, noise, and other unwanted signals. On the other hand, as C grows, we prefer to design as close to $\log_2 C$ as we can for obvious economic reasons. It would also be desirable if the design could incorporate features that enable the number of criteria of choice, as of identification to approach $\log_2 C$ on an historical or statistical basis.

It is important to recognize that a meaningful non-numeric data processing operation does not proceed from an algorithm that is based on exhaustion of all possibilities. This approach would be limited quickly by economic reasons as the number of objects to be processed (i. e., identified) increases. The alternative is to introduce an algorithm based on deductive operations.

An excellent working example is the problem of determining the structure of long chain molecules. These molecules are built up of sequences of amino acid, with certain rules (which are as yet not exhausted) of formation. A molecule currently being worked on by W. L. Duda, has a sequence of 250 amino acids comprising its structure, the number of different acids being 20.

Before we detail the information that is available to solve this problem, let us consider its dimension. If we have 20 acids to place, into 250 positions, assuming no empty position, then it is clear that we have a total of $20^{250} \sim 10^{325}$ different combinations of acids, and hence different molecules that could be constructed if we had no other information available.

Clearly, any method based on exhaustion is impossible. Moreover, the application of certain restraints, as we shall see, will sharply reduce the number of possible combinations of acids (i. e., molecules) to be considered by exhaustion. It is at this stage that a deductive operation becomes necessary.

Before we go on to a discussion of the deductive operation, let us consider some restraints that have been determined experimentally, and see to what extent they limit the number of possibilities.

A typical result obtained when the molecule is examined for structure is that the lefthand member of the chain is determined. From this we observe the following:

a. If this same acid may be replicated throughout the molecule, the number of possibilities becomes equal to or less than 20^{249}, which is a reduction by a factor of at least 20 from the original count of 20^{250}.

b. If this acid is unique, then the number of possibilities becomes $19^{249} \sim 10^{318}$, which is a reduction by a factor of 10^{7}.

It is clear that restraints such as these are insufficient: for an exhaustive procedure, both universes are effectively infinite.

Another empirical result is obtained from molecular weight determinations. No attempt is made to determine structure, but the relative weights of the constituents of the molecule are obtained, and so their relative number is determined. It can be readily shown that such a restraint is helpful but still insufficient. As an example, a distribution was assumed that was somewhat symmetric about the average, and the number of possible acids was computed to be 10^{110}, a reduction from the original by a factor of 10^{215}.

If we combine two restraints, e.g., that determined from molecular weights, and the determination of the lefthand constituent of the molecule (with replication of that constituent) we find that the further reduction in the number of possibilities is only another factor of 10: 10^{109}.

It seems reasonable to conclude that exhaustion procedures are not feasible; moreover, if there is no underlying theory or strategy, determining the restraints empirically itself becomes an exhaustion procedure.

Since it is reasonable to assume that the molecule is finite, a procedure that involves successively "chipping" away at one end of the molecule, and identifying the constituent so obtained is bound ultimately to result in determination of its structure. This procedure has the same status as an existence proof: one goes through it once, but having determined its validity, one searches for better procedures. The motives for this are sound: one must conserve time, money and equipment.

Less involved than the laborious procedure of "chipping" away at the molecule is the process of fragmentation. As a result of this procedure, a group of fragments of the molecule is obtained with a more or less random

distribution among these fragments of length, that is of number of constituents. These fragments can be separated according to lefthand identifying acid and according to composition.

If enough fragmentation has been produced, then sufficient raw data should be available from which the structure of the original molecule can be deduced. Moreover, the same restraints discussed above for the entire molecule apply to the fragments. Since many of these fragments are considerably smaller than the original, their universe of possibilities also shrinks, and deduction from these restraints becomes more feasible.

We consider the fragments to be classes, the elements of which are the constituent acids. The defining relations are identification of the lefthand acid and population of constituent acids in the fragment.

The two transitive operations with which deduction is performed are "lefthand location" and "class inclusion." To illustrate the first operation: if acid A is to the left of acid B, and acid B is to the left of acid C, then acid A is to the left of acid C. To illustrate the latter operation: if fragment, F_1, is included in fragment F_2, and if fragment F_2 is included in fragment F_3, then F_1 is included in F_3.

It is evident that fragments consisting of two, three, or a few acids as well as fragments lacking but two, three, or a few acids are most useful to start with, and that the most difficult set of fragments to work with are those that contain about half the positions of the original molecule.

For the molecule we have been discussing, with 250 positions, and 20 acids to be fitted, a set of 1000 fragments is being worked with. These fragments are sorted in rank of increasing number of acids after being separated into groups defined by the identified lefthand acid. After these operations have been completed, each group is then ordered under the two transitive relations: lefthand acid and class inclusion.

We note, that the sorting operation is itself an ordering in class inclusion, but both of these transitive operations are likely to produce <u>partially</u> ordered sets in view of the high order of replication of the acids.

An interaction among these lists is required in order to complete the deductive operation within a reasonable interval of time, or failing that, to indicate as soon as possible that insufficient data are at hand.

From a machine design point of view the interesting operation to mechanize is that of logical interaction among the ordered lists. Two objectives may be decided on: (a) to minimize bookkeeping; (b) to optimize flexibility. These objectives are somewhat contradictory in that a hardware

embodiment of a particular strategy may minimize bookkeeping for it, but at the same time considerably reduce the flexibility of the machine.

The essence of the logical interaction among the lists described above is that one starts with the shorter members of each list and progresses down each list as conclusions are obtained. We note that both sequential and parallel operations are called for, with non-destructive read-out of data from one list to be processed with another list. The same data are to be used elsewhere. Hence, modularity associated with the lists seems to be reasonable, and a further conclusion would be that the data be handled in a "multiprocess manner."

If we increase the dimensions of this problem, i.e., number of acids and number of sites, we can see that this will force the operation to be realized by a highly parallel machine, if only to obtain meaningful results within a reasonable interval of time.

By generalizing this problem in the sense of transforming "acid names" to "alphabet" and "sites within the molecule" to "finite string of symbols," we see that similar considerations apply to the analysis of data consisting of a finite batch of alphanumeric symbols that is presented in some arbitrary form. The transitive relations used will undoubtedly differ for each specific problem, and machine design, as well as instruction set must take this into account.

We now briefly consider another non-numeric problem: character recognition. Specifically, we discuss the problem of distinguishing a missile or missiles from an accompanying decoy or decoys. These entities are presented to the observer for a short interval of time, during which he must not only detect the objects, track them and obtain sufficient data for identification, but also decide on defense procedures that must take place in order to destroy the detected object if it is a missile.

A basic difference between this problem and problem described above is the dynamic nature of the detection problem. Furthermore, we have no a priori guarantee that all necessary data will be obtained in the time interval allotted to reception of data, so that action must be based on results that have various degress of certainty associated with them. What this amounts to is the calculation of a set of a posteriori probabilities from a set of a priori probabilities without really knowing whether Bayes' theorem is valid in these particular circumstances.

These factors aggravate the problem:

a. Time span in which data processing must be done.

b. Incomplete partitioning of identifying characteristics of missile vs decoy.

c. Small number of observations.

d. Subjective nature of a priori probabilities.

In order to examine more quantitatively the influence of the factors listed above (as well as some other aspects of the problem), a simple model of the detection problem was constructed, and a posterior probability was calculated. This follows:

a. Given an observed target, this target is either a missile or a decoy, exclusively:

$$T \equiv M \veebar D \tag{1}$$

b. we select a particular effect from a set of possible effects, and this effect is observed, or it is not:

$$E \equiv B \veebar W \tag{2}$$

c. we make a sequence of observations on the same target.

d. We assume the effect observed during a particular observation is independent (other than the target being the same) of effects observed during previous observations of this sequence, and the observed effect will have no influence on effects observed in subsequent observations of this sequence.

e. We are given the following a priori probabilities:

$$p(M) + p(D) = 1 \tag{3}$$

$$p(W/M) = \frac{p(M \cdot W)}{p(M)} \tag{4}$$

$$p(W/D) = \frac{p(D \cdot W)}{p(D)} \tag{5}$$

Where p(M) is the (subjective) a priori probability that the observed target is a missile, p(W/M) is the conditional probability, determined by experiment, that the observed effect will be W, knowing that the target is a missile. Similarly, p(W/D) is the conditional probability, also determined by experiment, that the observed effect will be W, knowing that the target is a Decoy.

From the equations in assumption e, above, we can calculate the posterior probabilities, such as p(M/W), for a single observation on the target. Thus, having observed a target, and noted the effect W, we obtain the conditional probability that the observed target is a missile from:

$$p(M/W) = \frac{p(M \cdot W)}{p(W)}$$

$$= \frac{p(M)\ p(W/M)}{p(M)\ p(W/M) + p(D)\ p(W/D)} \qquad (6)$$

Equation (6) is a statement of Bayes' theorem. Rewriting equation (6) in ratio form, we have:

$$p(M/W) = \frac{1}{1 + \frac{p(D)}{p(M)} \cdot \frac{p(W/D)}{p(W/M)}} \qquad (7)$$

It is somewhat easier to relate the ratios of equation (7) to our intuitive feeling for the problem in the sense of "what are the odds?"

Thus, $\frac{p(D)}{p(M)}$ states the a priori odds, and

$\frac{p(D)}{p(M)} \cdot \frac{p(W/D)}{p(W/M)}$ is a statement of the a priori odds, modified by the results of experimental (or heuristic) information.

We generalize equations (6) and (7) to take into account a sequence of observations on the same target, using, however, the assumption of randomness stated in assumption d, above.

Given a sequence of n observations, of which i will be W, and n - i, B, it can readily* be shown that:

$$p(M/n;\ i) = \frac{1}{1 + \frac{p(D)}{p(M)} \cdot \left(\frac{p(W/D)}{p(W/M)}\right)^i \cdot \left(\frac{p(B/M)}{p(B/D)}\right)^i \cdot \left(\frac{p(B/D)}{p(B/M)}\right)^n} \qquad (8)$$

is the expression for the posterior probability that the detected target is a missile when for a sequence of n observations, i of these observations yield the effect W. It is clear that

$$p(D/n;\ i) = 1 - p(M/n;\ i) \qquad (9)$$

for the same interpretation of n and i as above.

*See Appendix

Figure 1 shows the graphs of p(M/n; i) as a function of i for fixed n, with $\frac{p(D)}{p(M)}$ as a parameter.

The use of the a priori odds as a parameter emphasizes the influence that this subjective initial condition has on the prediction, and enables one to determine somewhat more quantitatively, a value of n, with included values of i, that will counteract extremes on the a priori odds.

We use the following values to construct the graph:

$$p(M) = 0.1,\ 0.5,\ 0.9$$

$$n = 10$$

$$0 \leq i \leq 10$$

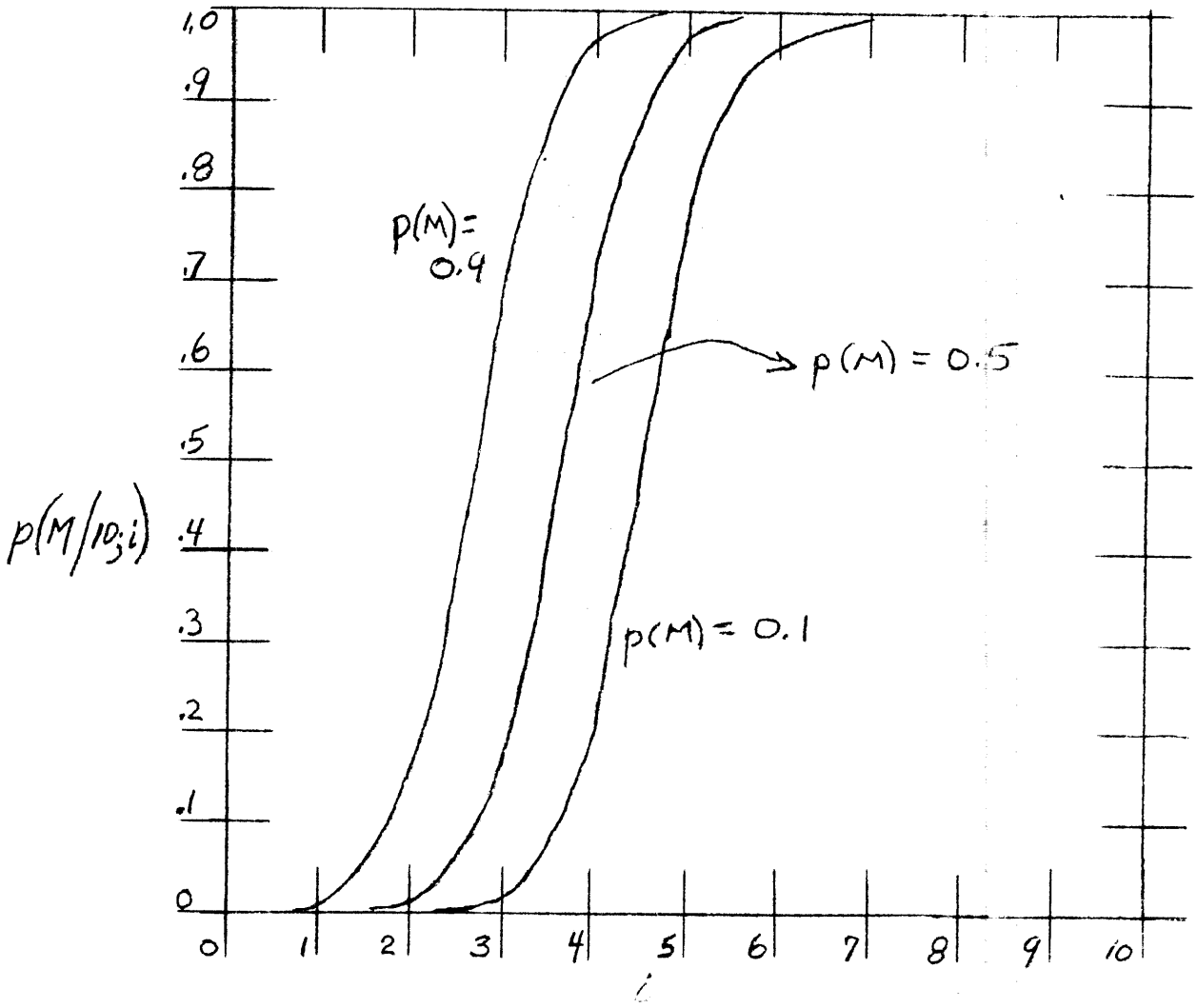
$$\frac{p(W/D)}{p(W/M)} = \frac{1}{5},\ \frac{p(B/M)}{p(B/D)} = \frac{2}{5}$$

Note that the graph shows three curves:

"Optimist's" curve	$p(M) = 0.1$
"Minimum knowledge" curve	$p(M) = 0.5$, and
"Pessimist's" curve	$p(M) = 0.9$.

It may be reasonable to assign the "pessimist's" a priori probability as the confidence value which the predicted probability obtained from the sequence of observations based on the "optimist's" a priori probability must exceed. The calculation, as shown, is clearly suggestive of an approach that may be followed.

The extension of this argument to many effects or to a quantized group from a continuous effect is relatively straightforward. A single effect is disjoint with all others, and arguments for independence can be established. Moreover, with different "sensors" operating, it can be seen that independent predictions can be made simultaneously, i.e., in parallel, with resultant checking.



POSTERIOR PROBABILITY OF M AS A FUNCTION OF NUMBER OF W OBSERVATIONS IN A SEQUENCE OF 10 OBSERVATIONS.

FIGURE 1

Posterior Probability of M as a Function of Number of W Observations in a Sequence of 10 Observations

	$p(M/10; i)$ FOR		
i	$p(M) = \frac{1}{10}$, $\frac{p(D)}{p(M)} = \frac{9}{1}$	$p(M) = \frac{1}{2}$, $\frac{p(D)}{p(M)} = \frac{1}{1}$	$p(M) = \frac{9}{10}$, $\frac{p(D)}{p(M)} = \frac{1}{9}$
0	$0.\epsilon$	0.0001	0.0009
1	0.00014	0.0013	0.016
2	0.0018	0.016	0.147
3	0.0228	0.17	0.647
4	0.221	0.72	0.958
5	0.78	0.97	1 - 0.0035
6	0.955	1 - 0.005	$1 - \epsilon$
7	1 - 0.0018	$1 - \epsilon$	$1 - \epsilon$
8	$1 - \epsilon$	$1 - \epsilon$	$1 - \epsilon$
9	$1 - \epsilon$	$1 - \epsilon$	$1 - \epsilon$
10	$1 - \epsilon$	$1 - \epsilon$	$1 - \epsilon$

$\epsilon \leq 10^{-4}$

Appendix

Derivation of Equation (8)

To derive the general expression for the posterior probability p(M/n; i), where n is the number of observations and i the number of observations in which W is the effect, we start with equations (1) through (5) and extend the argument from a single observation to two observations and then to more than two.

Consider the case of a sequence of two observations on the same target. Let the observed effect at time 1 be E_1 and the observed effect at time 2 be E_2. Then, considering each interval of time separately, and applying (2) we get

$$\begin{aligned} E_1 &= B_1 \forall W_1 \\ E_2 &= B_2 \forall W_2 \end{aligned} \qquad (2) \qquad \text{(A1)}$$

Furthermore, each of equations (3), (4), and (5) applies separately to each time interval. Thus, for example:

$$p(W_1/M) = \frac{p(M \cdot W_1)}{p(M)}$$

$$p(W_2/M) = \frac{p(M \cdot W_2)}{p(M)} \qquad (4) \qquad \text{(A2)}$$

But, the original equation (4) refers to an experimentally determined condition. Therefore, we state the equality of the same effect observed at different times for the same known target as

$$p(W_1/M) = p(W_2/M) = p(W_i/M) = p(W/M). \qquad (4) \qquad \text{(A3)}$$

A similar statement holds for (5):

$$p(W_1/D) = p(W_2/D) = p(W_i/D) = p(W/D). \qquad (5) \qquad \text{(A4)}$$

The aspect of equation (A2) that stresses the time independence of observed effect in terms of its relation to time of observation may be stated explicitly, eliminating the results of previous observations:

$$p(W_2/E_1 \cdot M) = p(W_2/M) = p(W/M). \qquad \text{(A5)}$$

We get (A6) by writing (A5) in terms of defining compound probabilities:

$$p(W_2/E_1 \cdot M) = \frac{p(W_2 \cdot E_1 \cdot M)}{p(E_1 \cdot M)} = \frac{p(W_2 \cdot E_1 \cdot M)}{p(M)\, p(E_1/M)} = p(W/M) \qquad (A6)$$

from which we solve for the compound probability:

$$p(W_2 \cdot E_1 \cdot M) = p(W/M)\, p(E_1/M)\, p(M) \qquad (A7)$$

To be explicit, we expand on E_1 and also add the equations of $E_2 = B_2$:

$$\begin{aligned}
p(W_2 \cdot W_1 \cdot M) &= p(W/M)\, p(W/M)\, p(M) = p(W \cdot W \cdot M) \\
p(W_2 \cdot B_1 \cdot M) &= p(W/M)\, p(B/M)\, p(M) = p(W \cdot B \cdot M) \\
p(B_2 \cdot W_1 \cdot M) &= p(B/M)\, p(W/M)\, p(M) = p(B \cdot W \cdot M) \\
p(B_2 \cdot B_1 \cdot M) &= p(B/M)\, p(B/M)\, p(M) = p(B \cdot B \cdot M)
\end{aligned} \qquad (A8)$$

It is clear that we get a similar set of equations for the compound probabilities involving D:

$$\begin{aligned}
p(W_2 \cdot W_1 \cdot D) &= p(W/D) \cdot p(W/D) \cdot p(D) = p(W \cdot W \cdot D) \\
p(W_2 \cdot B_1 \cdot D) &= p(W/D) \cdot p(B/D) \cdot p(D) = p(W \cdot B \cdot D) \\
p(B_2 \cdot W_1 \cdot D) &= p(B/D) \cdot p(W/D) \cdot p(D) = p(B \cdot W \cdot D) \\
p(B_2 \cdot B_1 \cdot D) &= p(B/D) \cdot p(B \cdot D) \cdot p(D) = p(B \cdot B \cdot D)
\end{aligned} \qquad (A9)$$

The extension of the results shown in (A7), (A8) and (A9) to a sequence of more than two observations is straightforward:

$$p(M \cdot \text{n observations}) = p(M) \cdot \prod_{j=1}^{n} p(E_i/M) \qquad (A10)$$

We now consider the posterior probability as shown in equations (6) and (7) for a single observation, and extend it to two observations specifically W_1 and W_2:

$$p(M/W_1 \cdot W_2) = \frac{p(M \cdot W_1 \cdot W_2)}{p(W_1 \cdot W_2)} \tag{A11}$$

$$= \frac{p(M \cdot W_1 \cdot W_2)}{p(M \cdot W_1 \cdot W_2) + p(\overline{M} \cdot W_1 \cdot W_2)}$$

$$p(M/W_i \cdot W_2) = \frac{1}{1 + \frac{p(\overline{M} \cdot W_1 \cdot W_2)}{p(M \cdot W_1 \cdot W_2)}} \tag{A12}$$

Substituting the appropriate equations from (A8) and (A9) into (A12) we get:

$$p(M/W_1 \cdot W_2) = \frac{1}{1 + \frac{p(\overline{M})}{p(M)} \frac{p^2(W/\overline{M})}{p^2(W/M)}} \tag{A13}$$

We now make use of (A10) and write the general expression for the posterior probability on M:

$$p(M/n \text{ observations}) = \frac{1}{1 + \frac{p(\overline{M} \cdot n \text{ observations})}{p(M \cdot n \text{ observations})}}$$

$$= \frac{1}{1 + \frac{p(\overline{M})}{p(M)} \frac{\prod_{j=1}^{n} p(E_i/\overline{M})}{\prod_{j=1}^{n} p(E_i/M)}} \tag{A14}$$

To write (A14) in more easily calculated form, we let i = the number of observations in which the effect observed is W:

$$0 \leq i \leq n$$

Then we get:

$$p(M/n; i) = \frac{1}{1 + \frac{p(\overline{M})}{p(M)} \frac{p^i(W/\overline{M}) \cdot p^{n-i}(B/\overline{M})}{p^i(W/M) \cdot p^{n-i}(B/M)}} \tag{A15}$$

which is readily cast in the form shown as equation (8):

$$p(M/n;\ i) = \frac{1}{1 + \frac{p(D)}{p(M)} \left(\frac{p(W/D)}{p(W/M)}\right)^i \cdot \left(\frac{p(B/M)}{p(B/D)}\right)^i \left(\frac{p(B/D)}{p(B/M)}\right)^n}. \qquad \text{(A16)}$$

A similar expression may be written for p(D/n; i).

Machine Organization
Technical Memo # 27

RESEARCH
CENTER

May 24, 1961

A Review of Some Large Scientific Calculations With Implications to Large Computer Organization

by

H. Hellerman

Introduction

This report is the result of a short survey of some scientific problems currently being run on large computers and the inferences which may be drawn from them in the design of a large, fast machine. This survey was not undertaken to be comprehensive, but rather to detect a pattern, if one existed, of trends in computer usage and their possible influence on machine organization.

One of the principal difficulties in performing this task is a meaningful definition of a "scientific problem." This difficulty exists because for machine organization considerations we are not interested so much in the origin of the problem, i.e., whether it arises in a scientific laboratory or a business firm, as much as the nature of the problem and the solution algorithms used to solve it. To an increasing extent, it is found that large problems are becoming more data-processing in nature as contrasted with purely numerical type of calculations. A prime example is the increasing use of simulation techniques.

This paper is in many ways cursory, since none of the problems cited were studied in depth by the author. It is hoped, however, that by bringing together in one place several diverse topics, the reader will be given a flavor for the direction being taken by large scientific calculations.

The Appendix is a description of some considerations entering into the choice of instruction and data word lengths for a large scientific machine. It also includes a table of instruction execution statistics for the 704 gathered by J. C. Gibson of IBM-PDL Lab., Poughkeepsie.

General Considerations

In this section a summary will be given of some salient features of scientific calculations. We begin by listing the features which have often been taken as indicative of a scientific computation. *

*In the following, comparatives imply a file maintenance problem as the basis of comparison

1. High frequency of arithmetic operations.

2. Long sequences of arithmetic operations on a small amount of data.

3. Uncertainty about the magnitude range of numbers appearing in the course of computation.

4. Program sequencing relatively independent of data and calculated results.

5. Low or at least programmer-predictable input-output traffic.

Needless to say, the above properties (especially 2, 4, 5) do not apply to all problems classed as "scientific"; in fact, one of the conclusions of this study is that modern large-scale programs from scientific applications more often depart from these properties than has been the case in the past, and this trend will doubtless continue into the future.

Feature 2 implies accessing the same data or intermediate calculations over and over again, usually in an orderly manner. This has been a principal impetus to the development and use of sophisticated indexing hardware. The uncertainty of number ranges appearing in long calculations has led to the universal use of floating point arithmetic in almost all modern large scale machines designed for scientific problems.

Importance of Solution Algorithm

When studying problems with a view towards identifying desirable computer characteristics, it is essential to consider not only the problem statement as it arises from the scientist who poses the problem, but also the algorithms used by the programmer to prepare the problem for machine solution. (The algorithm in turn often depends upon the characteristics of the machine available, e.g., the size of high speed storage relative to problem requirements, type of addressing, etc.) A simple illustration of the crucial importance of the algorithm may be seen by considering the elementary linear steady state heat-flow problem.

(1) $$\nabla^2 u = 0$$

$u(x_\beta, y_\beta, z_\beta)$ specified; where u is the temperature and the subscript β denotes "boundary." Using standard differencing techniques the problem may be stated as:

(2) $$A U = C U_B$$

U - an n-vector of non-boundary temperatures

A - an n x n matrix

U_B - an r-vector of boundary temperatures (prespecified)

C - an n by r matrix

The solution for U may in principle be obtained as

(3) $$U = A^{-1} C U_B$$

Thus, matrix inversion and matrix multiplication routines would appear to be essential operations for this type of problem. The fact of the matter is that this approach would almost never be used in large problems since the special properties of the A matrix, e.g., sparseness and symmetry can be used to effect very great savings in both required storage and time of solution. For example, the inversion calculation in Eq. (3) requires about n^3 multiplications and additions if done by the general matrix inversion programs. Storage of the A matrix requires n^2 locations. The solution of Eq. (2) by a reasonably efficient relaxation method only requires storage on the order of n locations and an operations count on the order of n multiplications and n additions per iteration. Although many iterations are required before the answer is obtained, this is usually far fewer than n^2.

Library Problems

Library problems here denote the standard set of problem types for which programs would be found in almost any installation doing scientific calculation. These include both short subroutines (e.g., calculation of square root, e^x, etc.) and larger problems of which several types are listed in Table 2. A library program usually is written to allow for many problem variations. This is often accompanied by some loss in efficiency compared to what is possible by recoding any specific problem. Library programs are used where a given problem does not occur often enough to justify an entire programming effort, or as a subroutine of much larger problems or where there is no property of the problem which can be used for constructing a more efficient program.

A list of six library-type problems are given in Table 2. We make the usual qualifying comments on the use of such a table, e.g., the incompleteness of the set of problems; different programs for the same problems exist having different timings and storage requirements, etc. Also to be noted is that a number of the programs listed use iteration methods. In these cases,

Table 2*

Some Library Problems

t_m = machine multiply time

t_a = machine add time

Problem	Notation	Data Storage	Running Time
Matrix Inversion (Direct Method)	m = matrix dimension	$2(m^2 + m)$	$2.5m^3\ t_m$
Characteristic Roots of Matrix (Jacobi Method)		$2m^2$	$10m^3\ (2t_a + t_m)$
Solution of $\frac{dy}{dx} = F(x, y)$ (Modified Predictor Corrector Method)	T = time to compute F(x, y) N = dimension of y vector (number of equations)	13N	$4T + 2N\ (6t_m + 18t_a)$
Eliptic Partial Differential Equation (Over-Relaxation)	N = number of interior mesh points	N	$3N\ (t_m + 2t_a)$ per iteration
Roots of Polynomial	N = degree of polynomial	10N	$120N^2\ (t_m + t_a)$
Linear Programming (Modified Simplex)	n = number of unknowns m = number of constraint relations t_d = division time	$2n^2 + 2n + mn$	$(m + 1)(n + 4m + 3)\ t_m + (m + 1)(n + 7m + 6)\ t_a + (3m + 2)\ t_d$ Note: 3m iterations usually necessary

*This table was compiled from data in Reference (1). Running time estimates neglect I/O time. In modern machines this can usually be overlapped with computation on these problems. Estimates assume large matrices.

since the total number of iterations is dependent upon the criterion for the residual, the time per iteration in itself is not completely indicative of solution time. In spite of the above qualifying remarks, the problems listed are among the most often-used and the programs presented are indicative of modern competent programming.

Steady State Mesh Problems

There are two principal sources of steady state mesh problems:

1. Electrical (and to some extent fluid, gas) flow problems.

2. Boundary value problems reduced to mesh form by the differencing process.

Recently, very general programs have been written and are running for solving electric network problems. Brannin[13] reviews some of those that use the Kron method of network solution. Although the steady state mesh problem may be viewed mathematically as one of solving simultaneous equations, the modern programs go much further. They accept a description of the network in topological form, i.e., a simple listing of which element is connected to which network node. The elements themselves are specified as to size. Active elements such as generators must be specified as to their position, type and magnitude. The program takes this rather descriptive information and solves the network for currents and voltages. Note that the step of writing the network equations is performed in effect by the computer. Brannin lists formulas and numbers for time of solution for this type problem. For a non-repetitive network, 704 running time using his most efficient method is given by

$$T = 3.2\ m\ (p - m)^2 \quad \text{milliseconds}$$

m = number of meshes in the network

p = number of node pairs in the network.

For m = 16,300 and p = 16,400, the 704 running time is about 234 hours. The same number of meshes and nodes in a repetitive network (such as would arise in a field problem) would require only 16 hours of 704 time. It should be remembered that often these problems form part of a study and the user is interested in running through the solution many times with changes of parameters.

The same kind of technique has also been applied to transient network problems[14]. The topological statement formulation remains as before. This problem is more complex than the steady state problem since it involves integration of differential equations.

Aside from floating point arithmetic, the most characteristic feature of the mesh problem is the transformation of the topological statements to the form useful by the rest of the program. Since the topological statements are bit matrices, facility for handling bit vectors in a flexible manner is most useful. Such facility is not present in the 7090; where obtaining the 1's count in a 100 bit vector requires a subroutine of instructions. STRETCH includes this operation in its instruction set, but the operation is comparatively slow proceeding 8 bits at a time.

Initial Value Problems

Most of the differential equations of mathematical physics are the result of bookkeeping relationships (e.g., conservation laws) applied to an infinitesmal region of space or matter. In order to solve these equations numerically, it is not possible to use the infinitesmal form; the differential equations must be replaced by difference equations. There is no unique manner of doing this; many differencing schemes exist and some of their properties will be discussed next.

The fundamental question in the numerical solution of partial differential equations is: how does the solution of the difference equations compare with the solution of the differential equations from which they are derived? Two specific properties of interest are summarized briefly.

1. Convergence - Consider a differential equation and a corresponding difference equation (note that there is not a unique difference equation for a given differential equation). The values of the dependent variable at the mesh points, obtained by solving the differential equation we denote by the vector u. The corresponding vector resulting from the solution of the difference equation is denoted by the vector U. The difference between u and U is called the truncation or discretization error. This error is dependent upon the size of the mesh separation h. If, in the limit as h approaches zero, U approaches u, then we say that the differencing scheme converges. As a practical matter, mere existence of convergence while comforting, is of little value since one cannot pass to the limit. Estimates for the truncation error exist as a function of the mesh constant h for a number of special cases. Absence of such a relation does not, however, inhibit attempts at problem solution; one can, for example, include tests for "sufficiently uniform" solution in the program.

2. Stability - It is possible that for a given differencing scheme and choice of mesh constants that a phenomenon known as instability may appear in the attempt at numerical solution. Consider a partial differential with two independent variables x, t. When selecting a differencing scheme one must choose two variable increments $\Delta x, \Delta t$. Now it may happen that although the differencing scheme selected is a convergent one, for certain choices of

$\Delta x, \Delta t$ the error may grow without limit as $n\Delta t$ gets larger. Moreover, this instability of the numerical calculation may even exist for very small values of $\Delta x, \Delta t$. The crucial factor here is the dependence of the error on the relation between Δx and Δt. To illustrate, consider the linear diffusion equation:

$$\nabla \frac{\partial^2 u}{\partial x^2} = \frac{\partial u}{\partial t} \tag{5}$$

A differencing scheme which may converge is

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = \nabla \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{\Delta x^2} \tag{6}$$

where the superscript denotes the increment in t and the subscript the increment in x. The condition for stability is(3)

$$\nabla \frac{\Delta t}{\Delta x^2} \leq \frac{1}{2} \tag{7}$$

To illustrate, suppose it is decided that a certain mesh spacing Δx is sufficient for the purposes to which we wish to put the solution. The stability criterion then forces us to choose Δt, the computation time increment, to be smaller than $\frac{\Delta x^2}{2\nabla}$, regardless of the separation of the points in time where we desire the solution.

Of course, Eq. (7) only holds for the parabolic equation (5). For hyperbolic equations the condition is that $\frac{\Delta t}{\Delta x}$ must be smaller than some constant.

The stability problem discussed above arises where explicit differencing methods are used. Eq. (6) is an example of an explicit differencing scheme since the value of U at the $n+1$ time step may be calculated explicitly from values of U at the n time step. Another type of differencing scheme which is termed implicit may also be used. An example of an implicit difference formulation for Eq. (5) is

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = \nabla \frac{U_{j+1}^{n+1} - 2U_j^{n+1} + U_{j-1}^{n+1}}{\Delta x^2} \tag{8}$$

In this equation, U_j, U_{j-1}, U_{j+1} all appear for the time step $n+1$, and these are all unknown at time step $n+1$. Hence, the system of equations of Eq. (8) (for all j) must be solved together at each time step (value of n). The advantage of the implicit differencing schemes is that they are always stable. The price one pays for this is the apparent increase in computation required to solve the simultaneous equations. This may, however, not be as bad as appears at first glance since one has greater freedom in choice of increment with the implicit method.

One use of a large machine will be to initial value problems of the type presently being done but with more internal points in the mesh. The fact that more internal points are used of course increases the operation count and memory requirement. In addition, however, the user is forced to increase the number of time steps he takes to satisfy the stability condition. Thus, the problem grows faster than linearly with increase in mesh size.

A brief description of the data processing aspects of a large Weather Research problem is given in Reference 8. This is a large initial value problem which is being programmed for STRETCH. Some boundary-initial value problems of interest to the AEC are discussed next.

AEC Problems

H. Kolsky discussed the following problems of current interest to the AEC at a meeting at Mohansic on January 19, 1961.

1. Neutron Diffusion
2. Neutron Transport
3. Hydrodynamics
4. Nearest Neighbor Problem
5. Monte Carlo Solution of Boundary Value Problem
6. Weather Fall-Out Prediction.

Some of the specifics of these are noted in Table 3.

The neutron diffusion and neutron transport problems are steady state mesh problems which are solved using relaxation techniques. The solution time is proportional to the number of mesh points, the number of iterations (= 100 in the table) and the inner-loop execution time. The figures in the table neglect input-output time. Most of this can be overlapped with compute time.

It is easy to visualize sizeable growth of these problems. For example, the figure of 8,000 for number of mesh points in the neutron transport problem represents a 3-dimensional mesh (20 x 20 x 20). Using a mesh 80 x 80 x 80 would require a solution time of over 42 hours and storage of 8.27×10^6 words on STRETCH.

The hydrodynamics problem referred to in the Table was a dynamics problem, i.e., it included energy storage effects and hence the solution

Table 3

Summary of Three AEC Mesh Problems (6)

Problem	Number of Points	Data Storage	STRETCH Time
Neutron Diffusion	125,000	10^6	28 hrs
Neutron Transport	8,000	1.28×10^5	40 min
Hydrodynamics	125,000 in space 100 in time	1.25×10^6	4 hrs

was a function of time. The problem involved the solution of the following types of equations using Lagrangian coordinates:

1. Conservation of mass

2. Conservation of momentum

3. Conservation of energy

4. Thermodynamic Equation of State

5. Force balance.

For security reasons, the details of the other problems were not as available as for the three summarized in Table 3. Some general remarks by Kolsky on these are as follows:

1. The weather fall-out prediction is a real-time problem and consists of estimating fall-out from available weather conditions and expected explosion energy, so that scheduling of nuclear explosions can be made.

2. The Monte Carlo technique is one which simulates particle behavior in a model of a physical problem. It is used to solve boundary value problems with particularly difficult boundary conditions.

In the Monte Carlo procedure, each particle's history is followed through its travels and records are kept of its interaction with matter, i. e., collisions, energy interchanges, direction of travel, etc. In the process, new particles may be created; thus, the exact number of records at any stage of computation is not known in advance. The output is obtained by statistical operations on the particle records. Since the Monte Carlo technique may be used as a simulator, its advantage lies with the fact that although one needs a definite picture of the phenomena under study and the statistics of the forcing functions, the solution algorithm (other than simulation itself) need not be available. It is easy to be deceived into undue enthusiasm over this procedure until it is realized that:

1. Much fundamental knowledge about the problem is still essential to set up the problem.

2. The solution does not lead to generalization or understanding about the underlying physics of the problem.

3. If simulation is done at the finest level of the process (thus requiring the least number of simplifying assumptions), the problem may be too big to be handled by any system in a reasonable time. Hence, a key factor in

reducing the problem to manageable size is "variance reduction" or techniques for performing fewer trials than the simple theory indicates is required for the same accuracy.

For the above reasons, many physicists consider the Monte Carlo technique a "last resort." However, it is sometimes the only way to get an answer.

When Monte Carlo methods are applied to the solution of AEC particle problems, desirable machine properties include:

1. Built-in square root
2. Ability to pack data - several numbers per word
3. High speed branch operations
4. Large memory.

Further Discussion of Monte Carlo Methods

The uses to which Monte Carlo methods have been applied may be divided into two categories:

1. A tool in solving mathematical problems
2. Simulation.

In the first of these, one usually has a choice of another method to do the same job. The question of the advantage of Monte Carlo methods in this case has been considered in H. Kahn's article in Ref. (1), p. 251. A quotation from this article says:

"On the whole though, it must be admitted that Monte Carlo has not shown up very well in competition with standard techniques when these techniques have been at all reasonable. It has been most useful where standard techniques have been inadequate. In this sense, it is a method of last resort."

To illustrate the use of Monte Carlo techniques on a mathematical problem, consider the evaluation of an integral (the area under a curve). In normalized form

$$a = \int_0^1 f(x)\, dx$$

where the value of the known function $f(x)$ lies between 0 and 1.

To find the area under the curve $f(x)$ by Monte Carlo methods, one constructs a long sequence of trials. Each trial, i, is performed as follows:

1. Generate two random numbers x_i, y_i between zero and one.

2. Compute $f(x_i)$

3. Update a tally counter by 1 if $f(x_i) \geq y_i$.

These steps are repeated N times for $i = 1, 2 \cdots N$.

4. The final result is the contents of the tally counter divided by N. This procedure may be justified by a geometric argument which interprets each (x_i, y_i) as a random point on the plane where $f(x)$ is plotted, in the region bounded by the conditions $0 \leq x \leq 1 \; ; 0 \leq f(x) \leq 1$. The area under $f(x)$ is proportional to the number of these points lying under $f(x)$. The accuracy of the computation (reciprocal of the standard deviation) is proportional to the square root of the number of trials. Hence, to get an improvement of accuracy by a factor k, the number of trials must be increased by k^2. The same technique is applicable to evaluation of multiple integrals as discussed in Ref. (1).

Another interesting application of the Monte Carlo method is to matrix inversion[1]. Here the effect of round-off error is often negligible, the statistical error is the major concern.

Monte Carlo Methods and Parallel Computation

The Monte Carlo technique can be easily adapted to parallel processing. This can be done by initiating operations for several trials simultaneously. In servicing each trial several program steps must be performed including random number generation, function generation, updating of statistical quantities, etc. These operations can be performed in different sets of hardware units which are specialized for efficient execution of the particular type of function. Each type of unit may also exist in multiplicity. The problem of controlling such an array of hardware with great logical flexibility must reside in a stored program. This stored program and the machine control to implement it should require that the program-logical interlocks and the hardware-demand interlocks impose as little a burden as possible on the programmer. This is only possible if the system control is designed from the start with this type of operation as a primary mode of system operation. For an example of the programming of such a system, reference (9) may be consulted.

Modelling and Simulation

This application is felt to be a fertile field for the application of a very large computer. Evidence to support this includes a recent estimate by a

West Coast customer (SDC) of 90% of a 7090 installation time devoted to simulation. Very ambitious models of organization of socio-economic, business and military complexes have been constructed or planned. These include a model of certain decision-making functions of the President of the U.S., a model of the U.S. electorate used by the Democrats in the last election.

Less exotic but perhaps of more economic interest is the job shop type simulator of which at least 3 are in use by IBM and others. It will suffice to describe one of these briefly, the one specified in Reference (12). This is a 704 program which simulates a Job Shop for the purpose of analyzing and improving its operation.

This model allows user specification of the following parameters (this is not a complete set).

1. Routing of orders through machines.

2. Time spent at each machine. Set-up time and processing time is specified separately in each order for each machine used. Processing time includes specified statistical variations.

3. Order arrival rate into the Shop.

4. Order release patterns to the machines for processing.

5. Dispatching Rules - Any one of 10 rules for queue exploitation may be specified in a simulation run.

Outputs of the program include:

1. Load analysis report giving an analysis of hours of machine loads to be expected in the shop in a given time period.

2. Shop performance report giving machine and manpower utilization during the simulation period.

3. Analysis of Queues - showing the number of orders arriving and departing from the waiting line of each machine. This is broken down by dollar value of the order. Waiting time of each order in each queue and the length of each line is also made available.

4. Completion patterns of orders.

5. Inventory - carrying cost report.

The fundamental feature of simulator programs is list-keeping. Simulators are therefore primarily data processing and only secondarily numerical calculation type problems.

Appendix

A. Instruction Word Length Considerations

One of the most fundamental ways to increase the speed of a machine is to reduce the number of fetches to memory for instructions. This can be done by shortening the instruction length so that several instructions can be packed into a single memory word. The most elementary case is to provide for 2 instructions per word as STRETCH does. A more imaginative approach is to separate out the operand designation and "Op Code" parts of the present single instruction and make them separate instructions. This scheme is especially useful if one can design the system to make extensive use of implicit addresses. One way to accomplish this is to have a push-down type of arithmetic unit. Here the automatic indexing to control the push-down storage area provides implicit addressing so that many operand designation bits can be avoided in the program. The program may now be considered a stream of operation codes and operand designations, the latter now being fewer than in the conventional machine, and the "Op Code" instructions do not waste bits on operand fields that are not used.

Another approach to the achievement of short instruction length is the use of some type of indirect addressing. Already in existing systems we see the extensive use of index registers for holding base addresses, with the address parts of instruction words being used for increments. This type operation depends upon the commonly observed fact that addresses referred-to by adjacent or nearly adjacent instructions have a high degree of redundancy, at least to the memory area to which they refer. Advantage can be taken of this by allowing a short address in the instruction stream and keeping base address in a few fast registers. If the redundancy assumption is valid, the loading of the registers with long addresses will take place relatively infrequently.

Among the machines which use the above principles are the Burroughs B-5000, the CDC 160, the Bull Gamma 60, the University of Illinois computer, and to some extent, the IBM 1401 in its "chaining" feature.

B. Data Word Length Considerations

The decision on the data word length of a computer involves many factors. Although this decision need not even be made if a variable word length is used, this solution implies a serial and hence slow processing system relative to what is attainable with fixed word length. Where speed is not the dominant factor as in the IBM 1620, variable data length may be a reasonable solution. On the other hand, if the natural data "word" length for a given problem is not the same as the machine word size, then a fixed word length machine may dissipate much of its speed advantage in the programmed unpacking-packing operations and/or the multiple precision arithmetic required

to handle the "natural" field length. STRETCH provides for both variable field length and fixed word operations in recognition of these considerations. In addition to arithmetic operations, the variable field length mode only is available for certain operations such as the Boolean connectives. This was apparently done for economy reasons, since the speed to be gained by making these available in parallel for the full 64 bit word rather than the 8 bit byte would be appreciable in certain problems.

If the decision is made in favor of fixed word length, the next question that arises is, how is this length to be determined? Here again there are many things to consider, including the size (or sizes) of the instruction words, the character (byte) size of the VFL unit of data, number of bits in I/O device bytes, etc. Very little can be stated in a formal way about these considerations. However, one guidepost has resulted from analysis and deserves some discussion.

Von Neumann and Goldstine have analyzed the problem of inverting large matrices[10]. Here, as the matrix gets larger, the number of computations required for inversion grows and with any finite word length, eventually the round-off error will yield a result of very poor accuracy. The largest matrix which can be inverted with expectation of reasonable results is given by

(A-1) $$n < .15\,\beta^{s/4}$$

or $$s > 4 \log_\beta \left(\frac{n}{.15}\right)$$

where n = limiting size of the matrix
β = number base used
s = number of digits required.

This equation evaluated for $\beta = 2$ (binary number system) is shown for the 701, STRETCH and the 7090. Only the bits that hold the fraction in the floating point word are considered in this table. According to this sole criterion, one would conclude that STRETCH word size is adequate while the 7090 word length is far too small for the size matrices which can be expected in a machine with a 32 k memory. The reasons for the 36 bit word of the 7090 can be traced to the 701 which did not have floating point hardware. The 36 bit data word length is more justifiable for the 701 with its fixed point operations and the smaller memory which was assumed in the original design. Of course, Eq. A-1 is based on a single problem, matrix inversion, and it would be naive to determine the word length of a machine based on this alone. On the other hand, this relation is useful as a bench-mark since matrix inversion is a non-trivial scientific calculation.

Table 1

Matrix Size - Word Length Relation
of Von Neuman & Goldstine

Machine	Bits in Fraction	Dimension of Largest Matrix
701	36	77
7090	27	16
STRETCH	48	614

C. Instruction Execution Statistics - Gibson's Mix

Mr. J. C. Gibson of Dept. 915 has conducted a study of instruction executions for the 704. He has dynamically traced through 8,000,000 executed instructions on a set of scientific problems which included numerical calculations, and logical calculations. Input/output routines were not included in these counts. Gibson's data is included in Table C1. Also included for ready reference is the number of memory cycles required for each 7090 instruction.

Gibson's figures are valuable because they are about all that is available. On the other hand, one should be careful not to overestimate their value. It must be remembered that the properties of the "mix" used may be quite different than those for a problem of interest. Also, instruction usage may differ considerably for the 7090 (relative to 704) because of the increased instruction set, indirect addressing, etc. It would be especially dangerous to apply the statistics to a machine outside of the 704-7090 family.

Nevertheless, Gibson's figures are important because they give a benchmark which when used properly can aid in the design of scientific machines.

Table C1 [15]

Instruction Execution Frequencies From Gibson Mix IV

For IBM 704

Operation	Frequency (percent)	Ave. 7090 Cycles	Weighted 7090 Cycles
Load and Store	38.0	2	76
Fixed Add	7.5	2	15
Compare	4.6	3	13.8
Branch (conditional types)	20.2	2	40.4
Floating Add	8.4	6.4	53.8
Floating Multiply	4.7	11	51.7
Floating Divide	1.8	13	23.4
Fixed Multiply	.8	11.6	9.3
Fixed Divide	.2	14	2.8
Shift	5.3	4	21.2
Logical	2.0	3	6.0
Misc.	6.5	2	13.0
	100.0		326.4

Bibliography

1. "Mathematical Methods for Digital Computers," Ralston & Wilf (Wiley).

2. "On the Design of A Very High Speed Computer," Report 80, University of Illinois, 1957.

3. "Difference Methods For Initial Value Problems," Richtmeyer (Interscience Publ. Co.)

4. "Finite Difference Methods For Partial Differential Equations," Forsythe (Wiley).

5. "A Mass Transfer Finite Difference Formulation Employing Crocco Variables," Moran and Scott Tech. Report 443, MIT Naval Supersonics Lab.

6. Meeting with H. Kolsky at Mohansic, Jan. 19, 1961.

7. "The Relation Between Kron's Method and the Classical Methods of Network Analysis," F. Brannin, Proc. WESCON, 1959.

8. "A Sketch of Weather Data Processing," L. R. Johnson, Dept. 475, M. O. Memo #20.

9. "Matrix Calculations on a Multiprocessor," G. C. Driscoll, Research Report, RC-417.

10. "Numerical Inverting of Matrices of High Order," J. Von Neuman and H. Goldstine, Bulletin American Mathematical Society, Nov. 1947.

11. "A Simulator for Time-Independent Models" by F. E. Franklin, E. Furth, S. A. Resnick, R. D. Villani, IBM Research Report RC-348.

12. "Improved Job Shop Management through Data Processing." General Information Manual, E 20-6071.

13. "The Relation Between Kron's Method and the Classical Methods of Network Analysis" by F. H. Brannin, Jr., 1959 WESCON Convention Record.

14. "The A Matrix, New Network Description," T. R. Bashkow, Trans. IRE PGCT, Sep. 1957.

15. "Scientific Mix" by J. C. Gibson, File Memorandum of Jan. 25, 1961, DSD.